

Lineamiento de Buenas Practicas en DynamoDB

Realizado por	Fecha	Firma
Cafu Junior Aguilar Baldeon	19/02/2024	
Revisado por	Fecha	Firma
Anibal Hugo Cipriano Carrascal		
Jonathan Franchesco Torres Baca		
Aprobado por	Fecha	Firma
Jose Jesus Salazar Silva		

1. Introducción

1.1 Objetivo

1.2 Alcance

2. Diseño de Claves en DynamoDB

2.1. Clave de Partición (PK)

2.2. Clave de Ordenación (SK)

3. Buenas Prácticas en Consultas

3.1 Uso de Query en lugar de Scan

Ejemplo: Incorrecto - Uso de Scan

Ejemplo: Correcto - Uso de Query

3.2 Uso de Prefijos en SK para Filtrado Eficiente

Ejemplo: Obtener datos del primer parámetro

Ejemplo: Obtener datos de cualquier parámetro

1. Introducción

1.1 Objetivo

Este documento establece los lineamientos generales para el uso eficiente de **Amazon DynamoDB** en todos los sistemas y aplicativos de la empresa **Atlantic City**. Su propósito es

garantizar un desempeño óptimo, escalabilidad, seguridad y una gestión eficiente de costos en el almacenamiento y recuperación de datos.

La aplicación de estas buenas prácticas permitirá:

- Consultas eficientes sin escaneos innecesarios.
- Diseño de claves optimizado para un acceso rápido a los datos.
- Minimización del consumo de recursos y reducción de costos.
- Mayor facilidad de mantenimiento y escalabilidad en los sistemas.

1.2 Alcance

Este documento es aplicable a todos los sistemas y aplicativos de la empresa **Atlantic City** que utilicen **Amazon DynamoDB** como base de datos. Define estándares y lineamientos obligatorios para el diseño de claves, patrones de consulta, optimización del rendimiento y monitoreo del sistema, asegurando la correcta implementación y mantenimiento de la base de datos en los desarrollos internos.

2. Diseño de Claves en DynamoDB

El diseño adecuado de claves nos evita **cuellos de botella y consultas ineficientes**.

2.1. Clave de Partición (PK)

Debe elegirse según los requerimientos del negocio, considerando que será el campo principal para búsquedas, conteos y segmentación de datos. Toda la estructura de la tabla se organizará en función de esta clave, agrupando los registros según su valor.

- **Formato:** "<CLAVE>|<identificador único>"
- **Ejemplo:** "IDJUGADOR|12345678"
- **Beneficios:**
 - Distribuir uniformemente los datos en las particiones.
 - Facilitar consultas directas sin **Scan**.
 - Evitar colisiones y congestión en una sola partición.

2.2. Clave de Ordenación (SK)

Es un complemento de la clave de partición, que permite diferenciar y categorizar los registros dentro de una misma partición. Su estructura facilita consultas jerárquicas, búsquedas eficientes y una mejor organización de los datos.

- **Formato:** "<CLAVE1>|<valor1>|<CLAVE2>|<valor2>|<CLAVE3>|<valor3>"
- **Ejemplo:** "MAQUINA|Maq-1231|CUPON|10DESC|PUNTOS|302"
- **Beneficios:**
 - Permitir consultas jerárquicas dentro de una misma clave de partición.
 - Facilitar búsquedas eficientes por prefijo (`begins_with`).
 - Optimizar la organización y recuperación de datos estructurados.

3. Buenas Prácticas en Consultas

3.1 Uso de `Query` en lugar de `Scan`

El uso de `Scan` no está permitido en entornos productivos, ya que recorre toda la tabla y puede generar un alto consumo de recursos.

En su lugar, se debe utilizar `Query` con `PK` y `SK` definidas.

Ejemplo: Incorrecto - Uso de `Scan`

Python

```
1 table = dynamodb.Table("Jugadores")
2
3 response = table.scan()
4 print(response["Items"])
```

C#

```
1 var request = new ScanRequest
2 {
3     TableName = "Jugadores"
4 };
5
6 var response = await client.ScanAsync(request);
7 Console.WriteLine(response.Items);
```

Node.js

```
1 const command = new ScanCommand({ TableName: "Jugadores" });
2
3 const response = await docClient.send(command);
4 console.log(response.Items);
```

Problema: Esta consulta escanea todos los registros, lo que afecta el rendimiento y los costos.

```

1 response = dynamodb.query(
2     TableName="Jugadores",
3     KeyConditionExpression="PK = :pk",
4     ExpressionAttributeValues={":pk": {"S": "IDJUGADOR|12345678"}}
5 )
6
7 print(response["Items"])

```

C#

```

1 var request = new QueryRequest
2 {
3     TableName = "Jugadores",
4     KeyConditionExpression = "PK = :pk",
5     ExpressionAttributeValues = new Dictionary<string, AttributeValue>
6     {
7         { ":pk", new AttributeValue { S = "IDJUGADOR|12345678" } }
8     }
9 };
10
11 var response = await client.QueryAsync(request);
12 Console.WriteLine(response.Items);

```

Node.js

```

1 const command = new QueryCommand({
2     TableName: "Jugadores",
3     KeyConditionExpression: "PK = :pk",
4     ExpressionAttributeValues: { ":pk": "IDJUGADOR|12345678" }
5 });
6
7 const response = await DynamoDBDocumentClient.from(client).send(command);
8 return response.Items;

```

Ventaja: Recupera solo los registros necesarios sin afectar la base de datos.

3.2 Uso de Prefijos en SK para Filtrado Eficiente

Para consultas de primer parámetro se recomienda usar el `begins_with()` en la clave de ordenación, para consultas del resto de parámetros puede utilizarse la función `contains()`.

Ejemplo: Obtener datos del primer parámetro

Python

```

1 response = dynamodb.query(
2     TableName="Jugadores",
3     KeyConditionExpression="PK = :pk AND begins_with(SK, :sk_prefix)",
4     ExpressionAttributeValues={

```

```

5         ":pk": {"S": "IDJUGADOR|12345678"},
6         ":sk_prefix": {"S": "MAQUINA|Maq-1231"}
7     }
8 )
9
10 print(response["Items"])

```

C#

```

1 var request = new QueryRequest
2 {
3     TableName = "Jugadores",
4     KeyConditionExpression = "PK = :pk AND begins_with(SK, :sk_prefix)",
5     ExpressionAttributeValues = new Dictionary<string, AttributeValue>
6     {
7         { ":pk", new AttributeValue { S = "IDJUGADOR|12345678" } },
8         { ":sk_prefix", new AttributeValue { S = "MAQUINA|Maq-1231" } }
9     }
10 };
11
12 var response = await client.QueryAsync(request);
13 Console.WriteLine(response.Items);

```

Node.js

```

1 const command = new QueryCommand({
2     TableName: "Jugadores",
3     KeyConditionExpression: "PK = :pk AND begins_with(SK, :sk_prefix)",
4     ExpressionAttributeValues: {
5         ":pk": "IDJUGADOR|12345678",
6         ":sk_prefix": "MAQUINA|Maq-1231"
7     }
8 });
9
10 const response = await DynamoDBDocumentClient.from(client).send(command);
11 return response.Items;

```

Ejemplo: Obtener datos de cualquier parámetro

En este caso buscaremos < CUPON > que esta en el segundo orden para el siguiente registro:

"MAQUINA | Maq-1231 | CUPON | 10DESC | PUNTOS | 302"

Python

```

1 response = dynamodb.query(
2     TableName="Jugadores",
3     KeyConditionExpression="PK = :pk",
4     FilterExpression="contains(SK, :cupon)",
5     ExpressionAttributeValues={
6         ":pk": {"S": "IDJUGADOR|12345678"},
7         ":cupon": {"S": "CUPON|10DESC"}
8     }
9 )
10
11 print(response["Items"])

```

```

1 var request = new QueryRequest
2 {
3     TableName = "Jugadores",
4     KeyConditionExpression = "PK = :pk",
5     FilterExpression = "contains(SK, :cupon)",
6     ExpressionAttributeValues = new Dictionary<string, AttributeValue>
7     {
8         { ":pk", new AttributeValue { S = "IDJUGADOR|12345678" } },
9         { ":cupon", new AttributeValue { S = "CUPON|10DESC" } }
10    }
11 };
12
13 var response = await client.QueryAsync(request);
14 Console.WriteLine(response.Items);

```

```

1 const command = new QueryCommand({
2     TableName: "Jugadores",
3     KeyConditionExpression: "PK = :pk",
4     FilterExpression: "contains(SK, :cupon)",
5     ExpressionAttributeValues: {
6         ":pk": "IDJUGADOR|12345678",
7         ":cupon": "CUPON|10DESC"
8     }
9 });
10
11 const response = await DynamoDBDocumentClient.from(client).send(command);
12 return response.Items;

```